



PROJECT REVIEW – 3

Recipe Sharing Platform with Meal Planner

Team 28 | Section 10 | Academic Year 2026–2027

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Thodeti Arun – 2520030456

Pavan – 2520090200

Project Overview

What it does

A web platform to share recipes, plan a weekly menu and generate a grocery list.

Why it is needed

Recipes are scattered across notes and chats, and weekly planning and shopping lists are done by hand.

Main functionality

Recipe sharing and search, ratings, saved recipes, meal planner, grocery list, nutrition info.

Technologies

React.js • Node.js + Express.js • MongoDB • Cloudinary • REST APIs

Front-End Development

Component-based UI

Reusable recipe cards, forms and rating stars.

Navigation

Page-to-page routing with React Router.

Forms

Add and edit recipe, login, register.

Data display

Recipe lists, details, planner grid.

User interaction

Search, filter, rate, save, plan.

API integration

REST calls through one Axios instance.

Frontend Modules / Features

1

Home and Search

2

Login and Register

3

Recipe Details

4

Add / Edit Recipe

5

Saved Recipes

6

Weekly Meal Planner

7

Grocery List

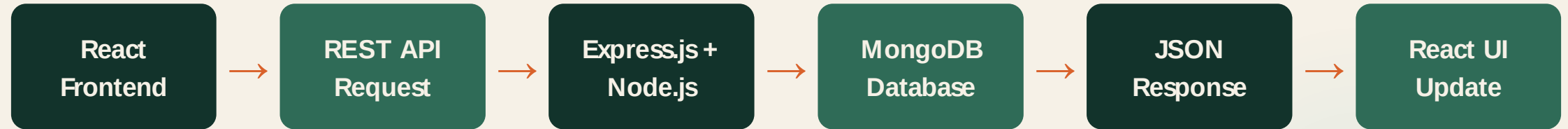
8

**Validation and
Feedback**

Frontend Screenshots



Frontend–Backend–Database Flow



- User actions in React trigger REST requests (GET, POST, PUT, DELETE).
- Express routes check the JWT, validate input and call Mongoose models.
- MongoDB returns documents, sent back to React as JSON.
- React updates its state and re-renders the screen.

Database Design

Document approach

Data stored as JSON-like documents in four collections.

Purpose

Store users, recipes, meal plans and grocery lists.

Data consistency

Mongoose schemas: required fields, enums, ranges, unique indexes.

Reduced redundancy

Plans reference recipes instead of copying them.

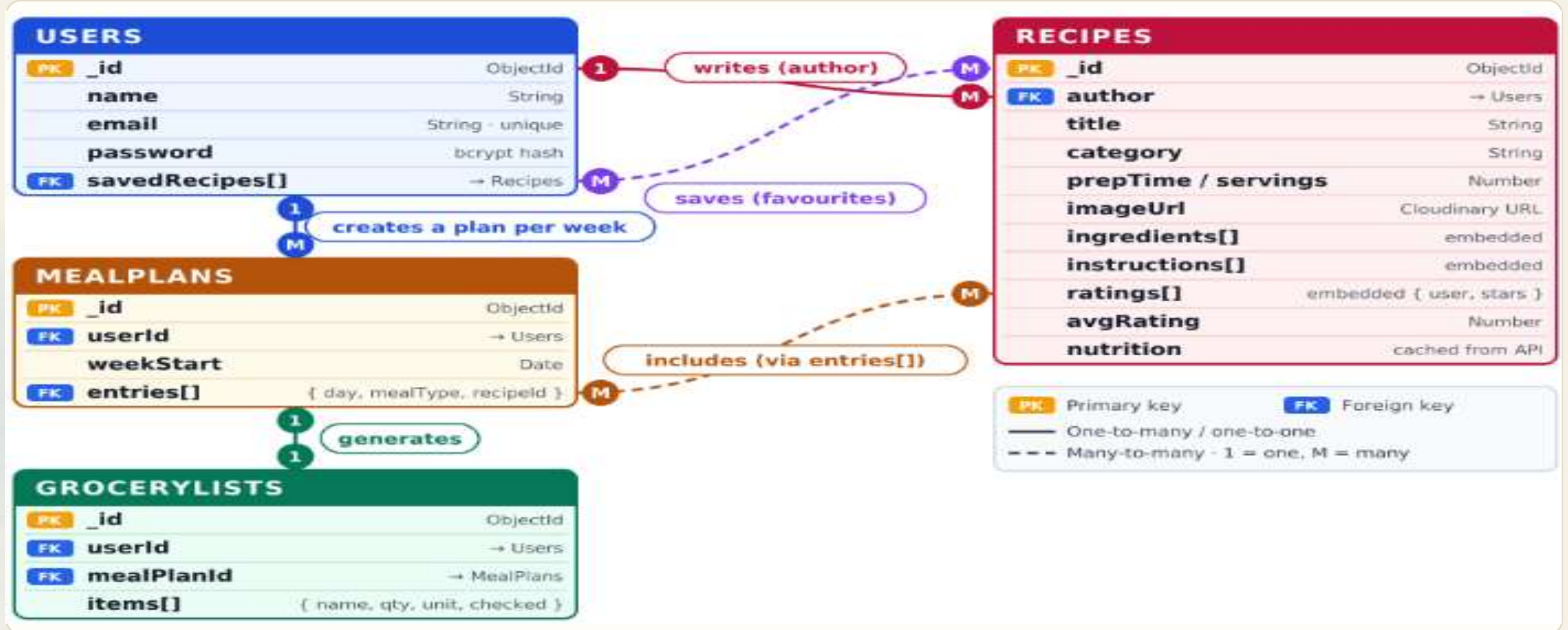
Relationships

Collections linked through ObjectId references.

Embedded data

Ingredients, steps and ratings live inside each recipe.

E R Diagram / Database Schema



Collections and Their Purpose

Collection	Purpose
users	Stores user accounts, hashed passwords and saved recipes
recipes	Stores recipe details, ingredients, steps, images, nutrition and ratings
mealplans	Stores each user's weekly plan as day and meal entries referencing recipes
grocerylists	Stores the merged ingredient list generated from a meal plan

Keys and Relationships

Primary Key

Every document has a unique `_id` (ObjectId).

Foreign Key (reference)

A field holding another document's `_id`, such as `author` or `recipeld`.

Referential Integrity

MongoDB does not enforce references; Mongoose and controllers check them.

Why it matters

References avoid duplicate data, so one recipe edit updates every plan.

Database Operations (CRUD)

CREATE

Register user, add recipe, save meal plan, generate grocery list.

READ

Search recipes, view details, load weekly plan and grocery list.

UPDATE

Edit own recipe, rate, change plan entries, tick items.

DELETE

Delete own recipe, remove plan entry, un-save recipe.

Sample Queries

1. Recipes in a category

```
Recipe.find({ category: 'Dinner' })
```

2. Weekly plan with recipe details

```
MealPlan.findOne({ userId, weekStart }).populate('entries.recipeId')
```

3. Top-rated recipes

```
Recipe.find().sort({ averageRating: -1 }).limit(5)
```

Current Implementation / Progress

Design and Planning

- Problem, scope and objectives defined
- Three-tier architecture designed
- 4 collections and relationships specified
- REST API endpoints planned
- Report and poster prepared

Implementation

- React frontend pages
- Express API with JWT authentication
- MongoDB connection and sample data
- Cloudinary and Nutrition API integration
- Postman testing of all endpoints

Conclusion and Next Steps

The platform combines a React frontend with a MongoDB database to let users share recipes, plan a week of meals and get a ready grocery list.

Next Steps

- Complete and polish the frontend pages
- Connect the API and load sample data
- Test every endpoint with Postman
- Add validation and better error handling
- Prepare a hosted demo



THANK YOU

Team 28 | Section 10 | Thodeti Arun and Pavan